IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

The Yenepoya institute of arts , science, commerce and management

Project – High Level Design

on

E-commerce CLI (E-commerce Application)

Student Name: Abhishek

Industry Mentor:Mr.shashank **Guided by: Ms.Yuktha**

Table of Contents

1. Introduction 3
1.1 Scope of the Document 3
1.2 Intended Audience 3
1.3 System Overview 3
2. System Design 4
2.1 Application Design 4
2.2 Process Flow 4
2.3 Information Flow 5
3. Components Design 5
4. Key Design Considerations 5
5. API Catalogue 5
6. Data Design 5
6.1 Data Model 5
6.2 Data Access Mechanism 6
6.3 Data Retention Policies 6
6.4 Data Migration 6
7. Interfaces 6
8. State and Session Management 8
9. Caching 8
10. Non-Functional Requirements 8
10.1 Security Aspects 8
10.2 Performance Aspects 8
11. References 9

1. Introduction

This document presents the High-Level Design for Abhishek Smart Store CLI, a comprehensive command-line e-commerce application with Flask backend and JSON persistence. The system enables users to browse products, manage shopping carts, and complete checkout transactions through an intuitive menu-driven interface with hybrid natural language processing support.

1.1 Scope of the Document

Covers architectural design of three-tier application comprising CLI frontend, Flask REST API backend, and JSON file-based data persistence. Includes system components, API specifications, data models, integration points, and non-functional requirements for the e-commerce platform.

1.2 Intended Audience

Software architects, backend developers, frontend CLI developers, quality assurance teams, system administrators, and technical stakeholders involved in implementation, deployment, testing, and maintenance of command-line e-commerce applications.

1.3 System Overview

The application follows client-server architecture where CLI frontend communicates with Flask backend via HTTP REST API. Users interact through menu-driven interface accepting both numbered selections and natural language commands. Backend manages product catalog, shopping cart state, and order processing with JSON file persistence as data store.

```
========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 6
Exiting Abhishek Smart Store.
(ecommerce) PS C:\Users\abhis\Desktop\Abhi-ecommerce\Abhi-ecommerce>
```

2. System Design

2.1 Application Design

Three-tier architecture with presentation layer implemented in Python CLI, business logic in Flask REST API, and data layer using JSON file storage. Frontend sends HTTP requests to backend endpoints, backend processes requests and mutates data store, responses formatted for CLI display. Separation of concerns enables independent development and testing of components.

2.2 Process Flow

User launches CLI which displays branded menu. User selects action through number or natural language command. CLI parses input and routes to appropriate flow handler. Handler makes HTTP request to Flask backend endpoint. Backend validates request, performs business logic, updates JSON data store. Backend returns JSON response. CLI formats response for display and returns to main menu loop.

```
(ecommerce) PS C:\Users\abhis\Desktop\Abhi-ecommerce\Abhi-ecommerce> python main.py
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 1

----------------------------------------
PRODUCT LIST
----------------------------------------
ID  Name          Price     Stock
1   Laptop        Rs.50000  10
2   Phone         Rs.20000  9
3   Headphones    Rs.2000   13
----------------------------------------
Select product (ID/name) or 'back': 2

----------------------------------------
PRODUCT DETAILS
----------------------------------------
Name     : Phone
Price    : Rs.20000
Stock    : 9
Delivery : 2 days
Specs    : 6GB RAM
----------------------------------------
1. Add to Cart
2. Back to Product List
Select action [1-2]:
```

2.3 Information Flow

Product catalog flows from JSON file to backend to CLI for display. User selections flow from CLI to backend as HTTP requests with product identifiers and quantities. Cart state maintained in backend and synchronized with JSON file. Stock levels updated atomically during add-to-cart operations. Checkout operation clears cart and persists order completion state.

3. Components Design

CLI Frontend Component: Menu rendering, input parsing with NLP support, HTTP client for backend communication, response formatting, flow orchestration. Flask Backend Component: REST API server, request validation, business logic execution, JSON file I/O operations, response generation. Data Storage Component: JSON file structure with products and cart arrays, atomic read-write operations, data persistence layer. HTTP Communication Layer: Request library for client-server communication, JSON payload encoding and decoding.

4. Key Design Considerations

Separation of Concerns: Frontend handles presentation, backend manages business logic, data layer ensures persistence. Hybrid Input Support: Menu numbers and natural language commands provide flexible user experience. Stock Validation: Atomic stock checks prevent overselling during concurrent operations. Graceful Error Handling: Network failures and invalid inputs handled with user-friendly messages. Stateless API: Backend endpoints designed as stateless operations with file-based state management. Single User Model: Current design optimized for single-user scenarios without authentication complexity.

5. API Catalogue

GET /products returns array of product objects with name, price, stock, delivery, specs. GET /cart returns array of cart items with product name and quantity. POST /add accepts JSON payload with product name and quantity, validates stock availability, decrements stock, adds to cart, returns success or error message. GET /checkout validates cart not empty, clears cart contents, persists changes, returns order confirmation or empty cart message.

6. Data Design

6.1 Data Model

Product Model: name string, price integer, stock integer mutable quantity, delivery string estimated time, specs string product specifications. Cart Model: product string product name reference, qty integer ordered quantity. JSON Root Structure: products array, cart array. All data persisted in single JSON file with pretty printing for readability.

6.2 Data Access Mechanism

Backend implements load function reading entire JSON file into memory and save function writing complete data structure back to disk. File path resolved relative to backend script location ensuring portability. Read operations occur at each API request, write operations occur after state mutations. No database or ORM layer required for current single-user scope.

6.3 Data Retention Policies

Product catalog persisted indefinitely with manual updates as needed. Cart state retained until explicit checkout or application restart. No automatic cleanup or archival mechanisms implemented. Order history not persisted beyond static profile display values. Future enhancements could implement order history database with timestamp-based retention policies.

6.4 Data Migration

Not applicable for current JSON file-based approach. Future migration to relational database would require schema definition, data transformation scripts, and backward compatibility handling for existing JSON data files.

7. Interfaces

Command Line Interface: User-facing menu system with hybrid input parsing, table-formatted product displays, cart summaries, and confirmation dialogs. REST API Interface: HTTP endpoints on localhost port 5000 accepting GET and POST requests with JSON payloads and responses. File System Interface: JSON file read-write operations with error handling for missing or corrupted files. Network Interface: HTTP communication between CLI client and Flask server using requests library.

```
Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]: 1

========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 2

----------------------------------------
YOUR CART
----------------------------------------
Item            Qty     Subtotal
Phone           x1      Rs.20000
----------------------------------------
TOTAL: Rs.20000

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]:
```

```
========================================
         ABHISHEK SMART STORE
========================================
1. Browse Products
2. View Cart
3. Checkout
4. Profile
5. Help
6. Exit

Select action [1-6]:
> Type number or command (e.g. 'show products')
> 5

----------------------------------------
HELP
----------------------------------------
Actions:
- show products / list / view items / 1
- cart / 2
- checkout / 3
- profile / 4
- help / 5
- exit / 6
----------------------------------------

Choose next step:
1. Return to Main Menu
2. Exit
Select action [1-2]:
```

8. State and Session Management

Application state maintained in JSON file serving as single source of truth. Backend API operates statelessly loading data on each request and persisting changes immediately. CLI maintains ephemeral state only for current menu navigation context. No user authentication or session tokens implemented. Cart associated with application instance rather than individual users. Future multi-user support would require session management and user authentication mechanisms.

9. Caching

No explicit caching layer implemented due to small data volumes and file-based storage. Each API request reads fresh data from JSON file ensuring consistency. Future optimizations could cache product catalog in memory since it changes infrequently while maintaining cart state synchronization. Python module import caching leveraged automatically by interpreter.

10. Non-Functional Requirements

10.1 Security Aspects

Input Validation: Product names and quantities validated before processing. Stock checks prevent inventory overselling. Localhost Binding: Flask server binds to 127.0.0.1 limiting network exposure. Request Validation: JSON payload structure validated before processing. Error Messages: Informative without exposing system internals. No Authentication: Current single-user model requires no authentication layer. File Permissions: JSON data file readable and writable by application process only.

10.2 Performance Aspects

Response Time: API endpoints respond within milliseconds for typical product catalogs under 100 items. File I/O: JSON file read-write operations complete within single-digit milliseconds. Network Latency: Localhost communication eliminates network latency concerns. Scalability: Current architecture suitable for demonstration and single-user scenarios, not production scale. Memory Footprint: Entire dataset loaded in memory acceptable for small catalogs. Concurrency: File-based storage not suitable for concurrent multi-user access without locking mechanisms.

11. References

Flask Official Documentation - https://flask.palletsprojects.com/. Python Requests Library Documentation. JSON File Format Specification. REST API Design Best Practices. Command-Line Interface Design Patterns. E-commerce System Architecture Principles. Roy Fielding Dissertation on REST Architecture. Python Official Documentation - https://docs.python.org/3/.